Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Цифрова схемотехніка

Лабораторна робота

№6

«ПРИСТРОЇ ДЛЯ ПЕРЕТВОРЕННЯ ЧИСЕЛ.
ТИПОВІ ВУЗЛИ КОМП'ЮТЕРА».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 9
Перевірив:
Нікольський С. С.

Київ – 2026

Лабораторна робота №6

Тема: «Пристрої для перетворення чисел. Типові вузли комп’ютера».

Мета: За допомогою середовища моделювання Modelsim розробити схему типового вузла комп'ютера, описавши її однією з мов опису апаратури (Verilog).

Хід роботи:

Варіант: 4209_{10}, у двійковому коді: $100000\ 1110001_2$
Знайдемо h: $h_7 = 1$, $h_6 = 1$, $h_5 = 1$, $h_4 = 0$, $h_3 = 0$, $h_2 = 0$, $h_1 = 1$

Таблиця 1 – Визначення варіанту

Номер групи	h_3	h_2	h_1	Розрядність суматора
ІО-Х2	*0*	*0*	*1*	*7*

Почнемо з реалізації 4-розрядного суматора

D:\Documents\Education\KPI\4Term\CompSchem\Rep\LR6\T1\ModelSimFiles\Kulikov_sum.v - Default

```
module my_sum (Ain, Bin, Ci, Sout, Co);
    input [6:0] Ain, Bin;
    input Ci;
    output [6:0] Sout;
    output Co;

    wire [6:0] C;

    bitsum sum1(Ain[0], Bin[0], Sout[0], Ci, C[0]);
    bitsum sum2(Ain[1], Bin[1], Sout[1], C[0], C[1]);
    bitsum sum3(Ain[2], Bin[2], Sout[2], C[1], C[2]);
    bitsum sum4(Ain[3], Bin[3], Sout[3], C[2], C[3]);

    assign Co = C[3];
endmodule

module bitsum (A, B, S, Cin, Cout);
    input A, B, Cin;
    output S, Cout;

    wire A, B, S, Res;
    wire c1, c2, Cin, Cout;

    xor(Res, A, B);
    and(c1, A, B);
    xor(S, Cin, Res);
    and(c2, Cin, Res);
    or(Cout, c1, c2);
endmodule
```

Рисунок 1 – Код для реалізації 4-розрядного суматора на базі 4-ьох однорозрядних

D:\Documents\Education\KPI\4Term\CompSchem\Rep\LR6\T1\ModelSimFiles\Kulikov_stSum.v - Default

```
module ref_sum (Ain, Bin, Ci, Sout, Co);
  input Ain, Bin, Ci;
  output Sout, Co;

  wire [3:0] Sout, Ain, Bin;
  reg [4:0] S;

  always @(Ain, Bin, Ci)
  S = Ain + Bin + Ci;

  assign Sout = S[3:0];
  assign Co = S[4];

endmodule
```

Рисунок 2 – Код для реалізації стандартного суматора на поведінковому рівні

```
Kulikov_testbench.v
module test_sum; // Top Level Testbench
      wire Ci,cm,cr;
      wire [3:0] Ain, Bin;
      reg [3:0] Ain_r, Bin_r;
      reg Ci_r;
      wire [3:0] res_my, res_ref;
      my_sum my_block (Ain, Bin, Ci,res_my, cm);
      ref_sum ref_block (Ain, Bin, Ci, res_ref, cr);
      initial
      begin
      $display("\t\t Time Ain Bin Ci res_my cm res_ref cr");
      $monitor($time,,,,,Ain,,,,,Bin,,,,,Ci,,,,,res_my,,,,,,,,cm,,,,,,,res_ref,,,,,,,cr);
      #400 $finish;
      end

      initial
      begin
      Ain_r = 1;
      #50 Ain_r = 5;

      #50 Ain_r = 1;
      #50 Ain_r = 5;
      #50 Ain_r = 1;
      #50 Ain_r = 5;
      #50 Ain_r = 1;
      #50 Ain_r = 5;
      end

      initial
      begin
      Bin_r = 2;
      #100 Bin_r = 10;
      #100 Bin_r = 2;
      #100 Bin_r = 10;
      end

      initial
      begin
      Ci_r = 1'b0;
      #200 Ci_r = 1'b1;
      end

      assign Ain = Ain_r;
      assign Bin = Bin_r;
      assign Ci = Ci_r;
endmodule
```

Рисунок 3 – Код для тестування

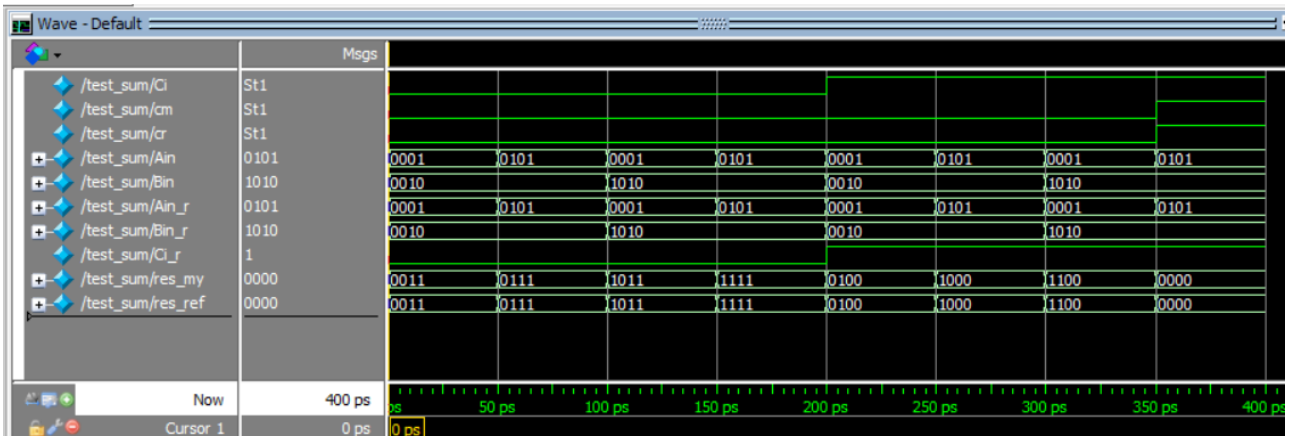

Рисунок 4 – Результат виконання симуляції

```
sim:/test_sum/res_ref
VSIM 4> run -all
#                Time Ain Bin Ci res_my cm res_ref cr
#                   0    1   2  0     3  0       3  0
#                  50    5   2  0     7  0       7  0
#                 100    1  10  0    11  0      11  0
#                 150    5  10  0    15  0      15  0
#                 200    1   2  1     4  0       4  0
#                 250    5   2  1     8  0       8  0
#                 300    1  10  1    12  0      12  0
#                 350    5  10  1     0  1       0  1
# ** Note: $finish    : D:/Documents/Education/KPI/4Term/CompSchem/Rep/LR6/T1/ModelSimFiles/Kulikov_testbench.v(15)
#    Time: 400 ps  Iteration: 0  Instance: /test_sum
```

Рисунок 5 – Результат виконання симуляції у консолі

Розробимо код, який реалізовуватиме семирозрядний суматор.

Kulikov_7b_refSum.v

```
module my_sum_7 (Ain, Bin, Ci, Sout, Co);
    input [6:0] Ain, Bin;
    input Ci;
    output [6:0] Sout;
    output Co;

    wire C4, C5, C6; // Внутрішні перенесення

    my_sum block4 (Ain[3:0], Bin[3:0], Ci, Sout[3:0], C4);

    bitsum sum4 (Ain[4], Bin[4], Sout[4], C4, C5);
    bitsum sum5 (Ain[5], Bin[5], Sout[5], C5, C6);
    bitsum sum6 (Ain[6], Bin[6], Sout[6], C6, Co);
endmodule

module ref_sum_7 (Ain, Bin, Ci, Sout, Co);
    input [6:0] Ain, Bin;
    input Ci;
    output [6:0] Sout;
    output Co;

    reg [7:0] S; // 8 біт для збереження суми та перенесення

    always @(Ain, Bin, Ci) begin
        S = Ain + Bin + Ci;
    end

    assign Sout = S[6:0];
    assign Co = S[7];
endmodule
```

Рисунок 6 – Код для реалізації семирозрядного суматора

Kulikov_7b_testbench.v

```
module test_sum_7; // tb 7-розрядного суматора
    wire Ci, cm, cr;
    wire [6:0] Ain, Bin;
    reg [6:0] Ain_r, Bin_r;
    reg Ci_r;
    wire [6:0] res_my, res_ref;
    my_sum_7 my_block (Ain, Bin, Ci, res_my, cm);
    ref_sum_7 ref_block (Ain, Bin, Ci, res_ref, cr);

    initial begin
        $display("\t\t Time \t Ain \t Bin \t Ci \t res_my \t cm \t res_ref \t cr");
        $monitor("%d \t %d \t %d \t %b \t %d \t\t %b \t %d \t\t %b", $time, Ain, Bin, Ci, res_my, cm, res_ref, cr);
        #400 $finish;
    end

    initial begin
        Ain_r = 7'd1;
        #50 Ain_r = 7'd25;
        #50 Ain_r = 7'd100;
        #50 Ain_r = 7'd127;
        #50 Ain_r = 7'd64;
        #50 Ain_r = 7'd10;
        #50 Ain_r = 7'd120;
    end

    initial begin
        Bin_r = 7'd2;
        #100 Bin_r = 7'd50;
        #100 Bin_r = 7'd127;
        #100 Bin_r = 7'd10;
    end

    initial begin
        Ci_r = 1'b0;
        #200 Ci_r = 1'b1;
    end

    assign Ain = Ain_r;
    assign Bin = Bin_r;
    assign Ci = Ci_r;

endmodule
```

Рисунок 7 – Код для реалізації тестування семирозрядного суматора

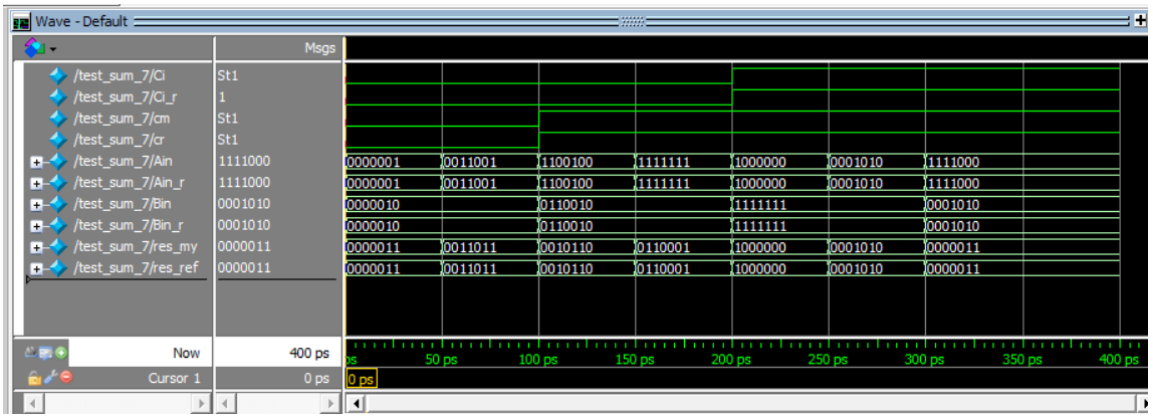

Рисунок 8 – Результат виконання симуляції

```
VSIM 8> run -all
#                Time    Ain     Bin     Ci      res_my          cm      res_ref         cr
#                   0      1       2      0          3           0          3           0
#                  50     25       2      0         27           0         27           0
#                 100    100      50      0         22           1         22           1
#                 150    127      50      0         49           1         49           1
#                 200     64     127      1         64           1         64           1
#                 250     10     127      1         10           1         10           1
#                 300    120      10      1          3           1          3           1
# ** Note: $finish    : D:/Documents/Education/KPI/4Term/CompSchem/Rep/LR6/T1/ModelSimFiles/Kulikov_7b_testbench.v(15)
#    Time: 400 ps  Iteration: 0  Instance: /test_sum_7
```

Рисунок 9 – Результат виконання симуляції у консолі

Як бачимо, суматори працюють правильно. При додаванні двох 7-розрядних чисел, які в результаті дають 7-розрядне число, ми бачимо точне значення, яке збігається з десятковим. Однак, коли результат потребує більше 7 розрядів, то спрацьовує вихід Co, який індикує перенесення зі старшого розряду. Як приклад, розберемо додавання чисел 100 і 50. У десятковій системі результат складатиме 150 (це більше за максимальне значення 127, а значить, ми не можемо повністю описати його у 7-розрядній сітці). Промоделюємо ситуацію додавання:

$$\begin{array}{r} 1100100\ (100) \\ +\ 0110010\ \ \ (50) \\ \hline 10010110\ (150) \end{array}$$

У нас з’являється додатковий 8-й розряд (одиниця зліва), і фізично вмістити його у 7-розрядну сітку ми не можемо. Ця одиниця йде на вихід перенесення Co. Залишок числа виводиться на шину результату. Це повністю збігається з тим, що ми маємо на часових діаграмах: при переведенні двійкового числа 0010110 у десяткову систему отримуємо 22. Значить, логіка переповнення відпрацьовує коректно, і симуляція виконана успішно.

Висновки:

Отже, в ході виконання лабораторної роботи було розроблено 7-розрядний суматор мовою Verilog. За допомогою ієрархічного підходу побудовано структурну модель на базі каскадного з'єднання простіших суматорів, а також створено еталонну поведінкову модель і testbench. Симуляція в ModelSim підтвердила повну коректність роботи пристрою, адже результати обох моделей ідентичні, і сходяться з ручними обрахунками. Усі файли проєкту завантажені на віддалений репозиторій GitHub за наступним посиланням: https://github.com/BronievM/KPI-DigitalSchem/tree/main/LR6/ModelSimFiles.